

Compare Amazon DynamoDB Reads with Amazon DAX

To Begin with the Lab

Summary of the Lab

In this lab, the same sample table was created in **Amazon DynamoDB** and then tested again through an **Amazon DynamoDB Accelerator (DAX)** cluster to compare read performance. The lab used a small test application to read the same item repeatedly so the difference between normal DynamoDB reads and cached DAX reads could be observed clearly. First, the application read data directly from DynamoDB and the latency was noted. Then, the same application was connected to DAX, where the first request pulled data from DynamoDB and later requests were served from cache. This showed how DAX improves repeated read performance without changing the DynamoDB table structure.

Understanding Amazon DynamoDB Accelerator (DAX)

Amazon DynamoDB Accelerator (DAX) is a fully managed in-memory cache for **Amazon DynamoDB**. It exists to reduce read latency when the same data is requested many times. Without DAX, every read goes directly to DynamoDB. With DAX, the first read fetches data from DynamoDB, but the next reads come from memory, which makes the application faster. This is useful for read-heavy workloads where the same records are accessed again and again.

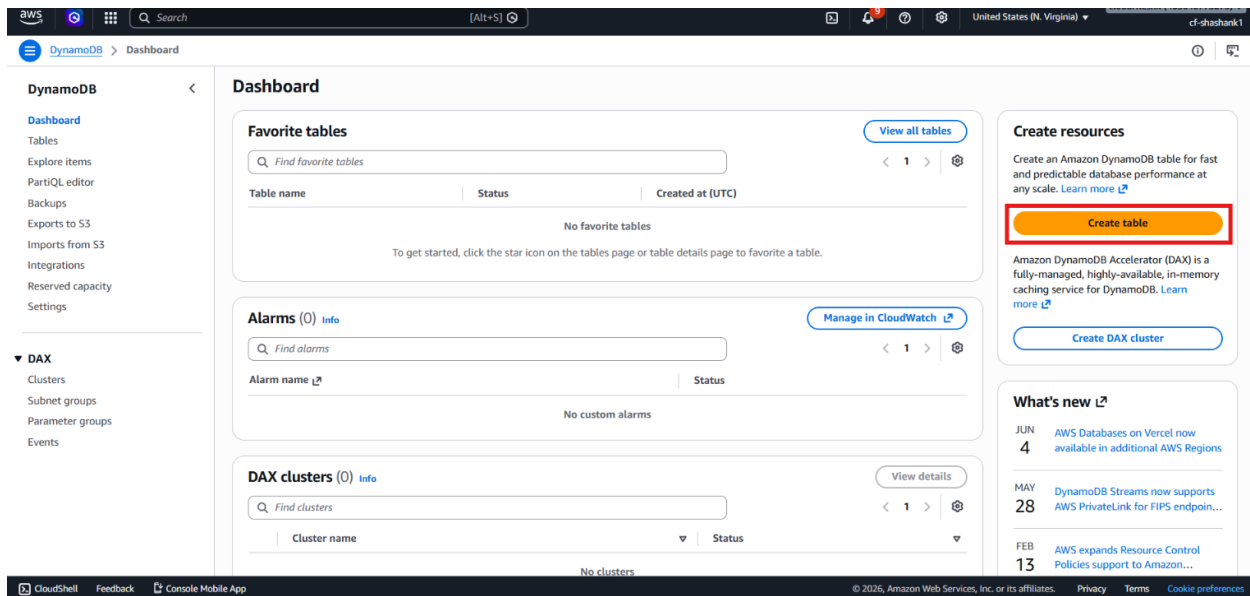
Example:

A product table stores item P101 for a laptop. When many users open the same product page, DynamoDB reads the item every time in the normal setup. With DAX enabled, the first request loads P101 into cache and the later requests return much faster from DAX.

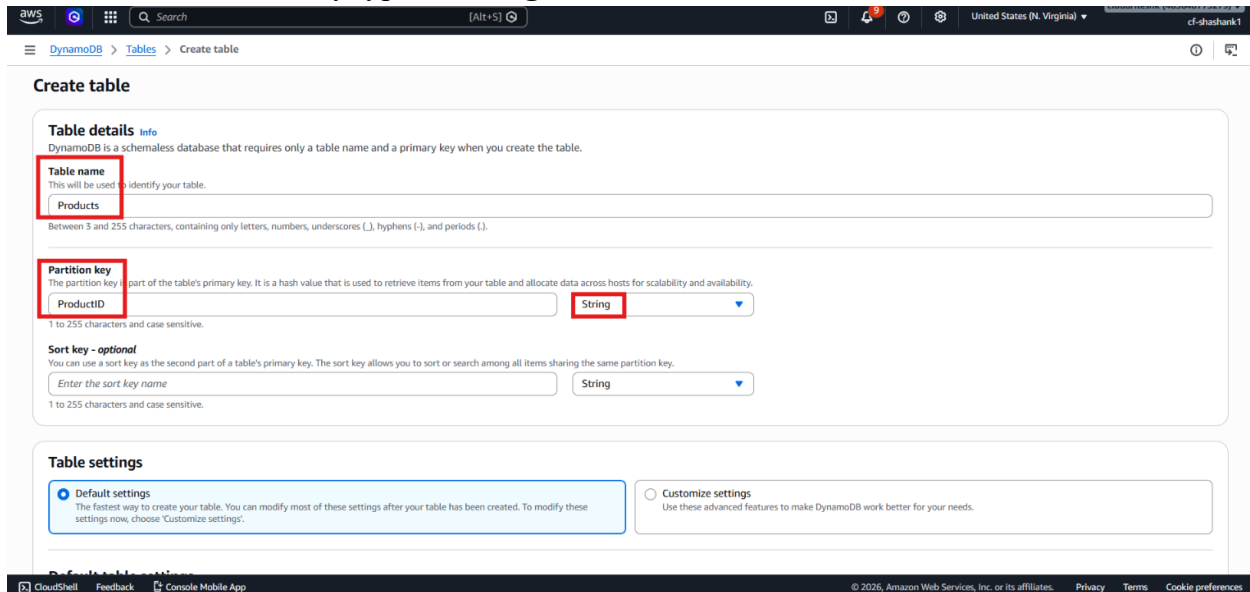
Lab Steps

Step 1 – Create the DynamoDB table

- Sign in to the **AWS Management Console**.
- Open **Amazon DynamoDB**.
- Click **Create table**.



- Enter the table name: **Products**
- Set the partition key, for example **ProductID**.
- Choose the correct key type as **String**.



- Create the table.

Step 2 – Insert sample data via script

- Open the newly created table.
- See there are no records.
- Here the sample script can be found in resources folder, do change your table name and region.

```
import pandas as pd
import boto3
```

```
from decimal import Decimal

TABLE_NAME = "Products"
REGION = "<YOUR_REGION>"
EXCEL_FILE = "<SAMPLE_EXCEL_FILE_PATH>"

df = pd.read_excel(EXCEL_FILE)

dynamodb = boto3.resource("dynamodb", region_name=REGION)
table = dynamodb.Table(TABLE_NAME)

with table.batch_writer() as batch:
    for _, row in df.iterrows():
        item = {
            "ProductID": str(row["ProductID"]),
            "ProductName": str(row["ProductName"]),
            "Category": str(row["Category"]),
            "Price": Decimal(str(row["Price"])),
            "Stock": Decimal(str(row["Stock"]))
        }
        batch.put_item(Item=item)

print("Upload complete.")
```

- Open VS Code or Terminal and run the following commands.
- Make sure to save the script and keep the sample excel file in same folder.

```
pip install pandas openpyxl boto3
pip install boto3 amazon-dax-client
python load_products.py
```

```
load_records_into_dynamodb.py X DynamoDB_DAX_Sample_Products.xlsx U
load_records_into_dynamodb.py > ...
Code Health: No application code detected for scoring
1 import pandas as pd
2 import boto3
3 from decimal import Decimal
4
5 TABLE_NAME = "Products"
6 REGION = "us-east-1" # change if your table is in another region
7 EXCEL_FILE = "DynamoDB_DAX_Sample_Products.xlsx"
8
9 df = pd.read_excel(EXCEL_FILE)
10
11 dynamodb = boto3.resource("dynamodb", region_name=REGION)
12 table = dynamodb.Table(TABLE_NAME)
13
14 with table.batch_writer() as batch:
15     for _, row in df.iterrows():
16         item = {
17             "ProductID": str(row["ProductID"]),
18             "ProductName": str(row["ProductName"]),
19             "Category": str(row["Category"]),
20             "Price": Decimal(str(row["Price"])),
21             "Stock": Decimal(str(row["Stock"]))
22         }
23         batch.put_item(Item=item)
24
25 print("Upload complete.")
```

```
PS E:\AWS Solution Architect\aws-git-cloned\architecting-on-aws> python aws_git_cloned\architecting-on-aws\load_records_into_dynamodb.py
Upload complete.
PS E:\AWS Solution Architect\aws-git-cloned\architecting-on-aws>
```

- Now refresh the table and you can see all 100 records.

The screenshot shows the AWS Management Console interface for a DynamoDB instance. On the left, the 'DynamoDB' sidebar is visible with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The 'Explore items' section is active, showing a list of tables: Employees, global_td-notes, hotelRoomAvailabilityTable, hotelRoomBookingTable, Products (selected), and td_notes_archive. The 'Products' table is selected, and the 'Scan or query items' section is displayed. The 'Scan' option is chosen, and the 'Table - Products' is selected. The 'Filters - optional' section is empty. The 'Run' button is clicked, and a status bar shows 'Stopped - Items returned: 50 - Items scanned: 50 - Efficiency: 100% - RCUs consumed: 1'. Below this, a table titled 'Table: Products - Items returned (50)' is shown, with columns for ProductID (String), Category, Price, ProductName, and Stock. The first row shows a product with ID 'P089', Category 'Home', Price '2499', ProductName 'Blanket', and Stock '20'.

Step 3 – Create the DAX cluster

- Open Amazon DynamoDB Accelerator (DAX).
- Click **Create cluster**.
- Enter the cluster name: **Products-Cluster**

- Choose the node type as **dax.t2.small**
- Select number of nodes as 2.

Cluster name
Provide a meaningful name that uniquely identifies your DAX cluster.
Products-Cluster
Must be between 1 and 20 characters; begin with a letter; contain only ASCII letters, digits, and hyphens; and not end with a hyphen or contain two consecutive hyphens.

Cluster description - optional
Enter description
Maximum 255 characters.

Node families
Choose the type of nodes to run in your clusters. All nodes in each cluster must be of the same type. You cannot modify the node types for a running DAX cluster.

Node type family
☐ All families
☒ t-type family
☐ r-type family

Node types (1/4)
Find node types

Node type	vCPU	Memory (GiB)	Network performance
☒ dax.t2.small	1	2.00	Low to Moderate
☐ dax.t2.medium	2	4.00	Low to Moderate

- Select the same VPC and subnet settings used for the app.
- But for now I will be selecting the already created **Subnet** and **Security Group**.

Network Type
To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.
IPv4

Subnet group
DAX will assign network addresses to your cluster nodes from the subnets included in this group. Subnets also determine the Availability Zones (AZs).
☒ Choose existing
☐ Create new
 dax-subnet-group (vpc-099792461c7d0a616)
 Last updated July 14, 2026, 21:17 (UTC+5:30)

Details of the chosen group
 Group description: Dax Subnet group
 Virtual Private Cloud (VPC) ID: vpc-099792461c7d0a616
 Subnets: 2 View

Access control
Security Group
 A security group acts as a firewall that controls network access to your DAX cluster.
 default (vpc-099792461c7d0a616)
 View in EC2 console
 To access the DAX cluster from your application, you must turn on inbound access on port 8111 for this security group, or port 9111 if encrypted in transit. For detailed instructions, see [Configure Security Group Inbound Rules](#)

- Choose the IAM Role for the Cluster.

Step 1 Choose cluster nodes
Step 2 Configure networks
Step 3 **Configure security**
Step 4 - optional Verify advanced settings
Step 5 Review and create

Configure security

IAM permissions
IAM Service role for DynamoDB access
Choose the IAM service role that your cluster nodes will assume when accessing your DynamoDB tables. The policy on this role controls which tables they can access, and which operations they can run.

☒ Choose existing
☐ Create new

DAXtoDyanamodb

Last updated July 14, 2026, 21:35 (UTC+5:30)
[View in IAM console](#)

Encryption

☐ Turn on encryption at rest
Protects your data while it is stored, at no additional cost. You cannot change this settings after the cluster is created. We recommend turning on encryption when possible.

☐ Turn on encryption in transit
Protects your data in transit, at no additional cost. Only the latest versions of the DAX client are compatible with encryption in transit. You cannot change this settings after the cluster is created. We recommend turning on encryption when possible.

[Cancel](#) [Previous](#) [Next](#)

- Create the cluster and make sure the status becomes Available.
- Note down the **Cluster Endpoint**.

DynamoDB
Dashboard
Tables
Explore Items
 PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings

▼ DAX
Clusters
Subnet groups
Parameter groups
Events

Clusters (1/1)
Search clusters
products-cluster

products-cluster
[Delete cluster](#)

[Overview](#) [Monitor](#) [Events](#) [Settings](#)

General information
[Edit description](#)

Description -

Status
Available
No active alarms

Cluster endpoint
dax://products-cluster.v10te.dax-clusters.us-east-1.amazonaws.com

Amazon Resource Name (ARN)
arn:aws:dax:us-east-1:cache/products-cluster

Nodes

Node type	vCPU per node	Memory per node
dax.t2.small	1	2 GiB

Nodes (2)
[Reboot](#) [Delete](#) [Add nodes](#)

Node ID	Status	Availability Zone	Created time	Parameter group status
products-cluster-a	Available	us-east-1a	July 14, 2026	in-sync
products-cluster-b	Available	us-east-1b	July 14, 2026	in-sync

Step 4 – Run the scripts to see read time

- Now run the following files but make sure to configure the region and table names.
- For dax_read.py make sure to properly configure the endpoint, replace it via your own Dax endpoint.

```
dax_read.py
dynamodb_read.py
```

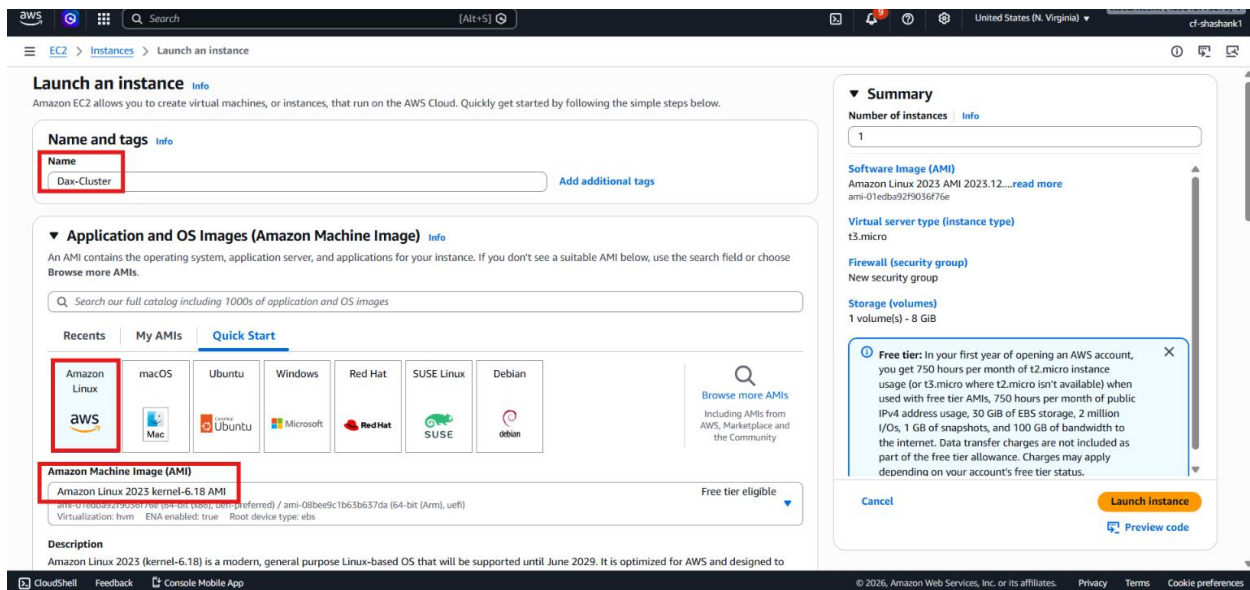
- Output for DynamoDB.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS E:\AWS Solution Architect\aws-git-cloned\architecting-on-aws> pip install boto3 amazon-dax-client
Installing collected packages: antlr4-python3-runtime, amazon-dax-client
Successfully installed amazon-dax-client-2.0.6 antlr4-python3-runtime-4.13.2

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS E:\AWS Solution Architect\aws-git-cloned\architecting-on-aws> python dynamodb_read.py
DynamoDB Time: 29.1354 seconds
```

Step 5 – Install required Libraries and Setup

- Open Amazon EC2.
- Launch a Linux instance such as Amazon Linux with name as **Dax-Cluster**.



- Allow **HTTP traffic** and select proper **Key pair**.
- Connect to the instance using SSH command as shown and run it in Command Prompt.

```
ssh -i "<YOUR_KEY_PAIR>" ec2-user@<YOUR_PUBLIC_IPV4_ADDRESS>
```

[illegible]

- Switch to root user by running the “sudo -i” command.
- Run the following commands to install python and required packages.

```
sudo dnf update -y
sudo dnf install python3-pip git -y
pip3 install boto3
pip3 install amazon-dax-client
```

```
C:\Users\Shashank\Downloads>ssh -i "June-2026-us.pem" ec2-user@3.80.216.190
The authenticity of host '3.80.216.190 (3.80.216.190)' can't be established.
ED25519 key fingerprint is SHA256:KREAUuWtQVbnR/7fz2/Fn92bAXznECL+OkHylg5Y5L4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.80.216.190' (ED25519) to the list of known hosts.
```



```
# Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
```

```
[ec2-user@ip-172-31-31-241 ~]$ sudo dnf update -y
sudo dnf install python3-pip git -y
pip3 install boto3
pip3 install amazon-dax-client
```

- Now import the modules and exit python.
- Run these commands line by line, do not copy paste.

python3


```
import boto3
from amazon dax import AmazonDaxClient
exit()
```

```
[root@ip-172-31-31-241 ~]# python3
Python 3.9.25 (main, Jun 17 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import boto3
m amazon dax import AmazonDaxClient
>>> from amazon dax import AmazonDaxClient
>>> exit()
[root@ip-172-31-31-241 ~]#
```

- Now Configure AWS Credentials by running this command and providing required details.

```
aws configure
```

```
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system packa
ge manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
[root@ip-172-31-31-241 ~]# python3
Python 3.9.25 (main, Jun 17 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import boto3
m amazon dax import AmazonDaxClient
>>> from amazon dax import AmazonDaxClient
>>> exit()
[root@ip-172-31-31-241 ~]# aws configure
AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: us-east-1
Default output format [None]: json
[root@ip-172-31-31-241 ~]#
```

- Now create the Python file as **dax_test.py**

```
nano dax_test.py
```

- Paste the required code and save it by pressing: Ctrl + O, Enter, Ctrl+X

```
root@ip-172-31-31-241:~
GNU nano 8.3 dax_test.py Modified
from amazon dax import AmazonDaxClient
import boto3
import time

client = AmazonDaxClient(
    region_name="us-east-1",
    endpoints=["dax.us-east-1.amazonaws.com"]
)

start = time.perf_counter()

for i in range(100):
    client.get_item(
        TableName="Products",
        Key={
            "ProductID": {"S": "P001"}
        }
    )

end = time.perf_counter()

print(f"DAX Time: {end-start:.4f} seconds")

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-G Copy
```

- Now run the file and note the output

```
python3 dax_test.py
```

```

>>> import boto3
m amazondax import AmazonDaxClient
>>> from amazondax import AmazonDaxClient
>>> exit()
[root@ip-172-31-31-241 ~]# aws configure
AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: us-east-1
Default output format [None]: json
[root@ip-172-31-31-241 ~]# nano dax_test.py
[root@ip-172-31-31-241 ~]# python3 dax_test.py
DAX Time: 0.4199 seconds
[root@ip-172-31-31-241 ~]#

```

Step 6 – Compare the results

- Review the read latency from both tests.
- Confirm that the first DAX read is slower than later reads.
- Confirm that repeated reads from DAX are faster than direct DynamoDB reads.

Verification

- Verified that the DynamoDB table was created successfully.
- Verified that sample records were inserted into the table.
- Verified that the application could read data directly from **Amazon DynamoDB**.
- Verified that the **Amazon DAX** cluster was created successfully.
- Verified that the application connected to DAX correctly.
- Verified that the first DAX request fetched data from DynamoDB.
- Verified that later DAX requests returned data faster from cache.
- Verified that DAX improved repeated read performance for the same sample table.